An Empirical-Based Security Benchmarking Framework for Container Confinement

by

Huzaifa Patel

A thesis submitted to the Faculty of Graduate and Postdoctoral Affairs
in partial fulfillment of the requirements for the degree of

Master
in
Computer Science

Carleton University
Ottawa, Ontario

© 2025 Huzaifa Patel

Nomenclature

VM virtual machine. 2

Chapter 1

Background

The existance of cloud computing can be attributed to virtualization, which is a technology that makes it possible for multiple different operating systems (OSs) to run concurrently, and in an isolated environment on the same hardware. How does virtualization achieve this? It makes use of a machines hardware to support the software that creates and manages virtual machines (VMs). A virtual machine (VM) is a virtual environment that provides the functionality of a physical computer by using its own virtual central processing unit (VCPU), memory, network interface, and storage. The software that creates and manages VMs is formally called a hypervisor or virtual machine monitor (VMM). The virtualization marketplace is comprised of four notable hypervisors, which are: (1) VMWare, (2) Xen, (3) Kernel-based Virtual Machine (KVM), and (4) Hyper-V. The operating system running a hypervisor is called the host OS, while the VM that uses the hypervisor is called the guest OS. To maintain consistency within this proposal, we will use the word "hypervisor" when referring to virtualization software. Hypervisors can be divided into two types, depending on where they are located on the machine: (1) type 1 and (2) type 2.

1.0.1 Type 1 Hypervisor

Type 1 hypervisors run directly on physical hardware to create, control, and manage VMs, and do not require support from the host OS. Type 1 hypervisors are also called native or bare-metal hypervisors. The first hypervisors, which IBM developed in the 1960s were type 1 hypervisors [?]. Examples of type 1 hypervisors include, but are not limited to Xen, KVM, VMware ESX, and Microsoft Hyper-V [?].

1.0.2 Type 2 Hypervisor

Type 2 hypervisors (also called hosted hypervisors) are installed on the OS (Windows, Linux, MacOS), similar to how computer programs are installed onto an OS. In other words, a type 2 hypervisor runs as a process on the host OS. Type 2 hypervisors abstract guest OSs from the host OS by introducing a third software layer above the hardware, as shown in figure 2.1. Examples of type 2 hypervisors include but are not limited to VMware Workstation, VirtualBox, and QEMU [?].

1.0.3 Problems With Type 1 & Type 2 Hypervisor Classifications

Although the definitions of type 1 and type 2 hypervisors are widely accepted, there are gray areas where the distinction between the two remain unclear. For instance, according to its creator RedHat, KVM is implemented and deployed using two Linux kernel modules that effectively convert the host Linux OS into a type-1 hypervisor [?]. At the same time, KVM can be categorized as a type 2 hypervisor because KVM VM's are standard Linux processes that are competing with other Linux processes for CPU time given by the Linux kernel's native CPU scheduler [?]. Due to disagreements and vagueness in the definitions of type 1 and type 2 hypervisors, two new classifications were defined: (1) native hypervisors and (2) emulation hypervisors [?].

1.0.4 Native Hypervisor

Native hypervisors are hypervisors that push VM guest instructions directly to the physical machines CPU, by using virtualization extensions like Intel VT-x. [**?**]. Examples of Native hypervisors include but are not limited to Xen, KVM, VMware ESX, and Microsoft HyperV.

1.0.5 Emulation Hypervisor

Emulation hypervisors are hypervisors that emulate every VM guest instruction using software virtualization [**?**]. Emulated guest instructions are very easy to trace because all guest VM instructions are trapped to the hypervisor. Examples of emulation hypervisors include but are not limited to QEMU, Bochs, early versions of VMware-Workstation, and VirtualBox [**?**].

1.0.6 Why do we have run Docker with sudo?

By default, Docker's daemon (dockerd) runs as root, and access to it is controlled through the Unix socket:

Copy Edit /var/run/docker.sock This socket is owned by root and typically part of the docker group. If your user isn't in the docker group, you'll need sudo to interact with it.

Why Docker Needs Elevated Privileges: Container Isolation Requires Kernel Features:

Managing cgroups, namespaces, mounting filesystems, setting up virtual networks—all require elevated permissions.

By Default, Docker Protects Access to Its Control API:

docker.sock is a privileged control interface that allows full system control (you can effectively become root via it).

1.0.7 YAMA

Yama is a Linux Security Module (LSM) that adds additional restrictions to the ptrace system call, which is commonly used for debugging and process tracing. While ptrace is a powerful tool for legitimate uses like debuggers and profilers, it can also be abused by malicious processes to interfere with or spy on other running processes. Yama was introduced to mitigate these risks by controlling and limiting which processes can use ptrace on others, providing a simple but effective hardening mechanism against certain types of attacks.

The core of Yama's functionality revolves around the /proc/sys/kernel/yama/ptrace_scope setting, which controls how restrictive the kernel is regarding ptrace access. The available levels range from 0 to 3, each enforcing progressively stricter controls. With ptrace_scope=0, the system behaves traditionally, allowing any process to ptrace another process owned by the same user if the target is marked as dumpable. At 1, only parent processes can trace their children, limiting the risk of arbitrary tracing. Level 2 requires a process to have CAP_SYS_PTRACE capabilities (effectively root-level permissions) to trace any process, and level 3 completely disables ptrace system-wide, locking down the system from process tracing even by privileged users.

Yama is especially valuable in multi-user systems or environments where processes run under the same user account but should remain isolated from one another, such as in containerized or sandboxed environments. While it is a relatively lightweight security module compared to more comprehensive systems like SELinux or AppArmor, it adds a meaningful layer of defense against local privilege escalation attacks. However, it's important to balance its restrictions with the need for debugging and profiling tools that rely on ptrace. In production environments focused on security, higher ptrace_scope values (such as 2 or 3) may be appropriate, while development systems typically keep it at 0 or 1 to allow standard debugging workflows.

Yama does not enforce its 'ptrace_scope' restrictions when attempting to trace host

processes from within a container because Yama's enforcement is limited to the current process visibility defined by PID namespaces. In most container environments, the containerized process runs in an isolated PID namespace and cannot see or interact with host processes unless explicitly configured using options like '–pid=host'. Even when a container is started with '–pid=host', allowing it to see host processes, Yama primarily governs interactions within the same namespace context and is not designed to enforce security policies across different namespaces.

Additionally, for Yama to restrict tracing of host processes, the containerized process would not only need visibility of host PIDs but also require sufficient privileges, such as 'CAP_SYS_PTRACE', in the host's user namespace. In practice, other security mechanisms like AppArmor, SELinux, and seccomp often prevent such privilege escalation before Yama's policies even apply. As a result, Yama's effectiveness is primarily confined to preventing ptrace interactions between processes that are already within the same visibility scope, rather than serving as a cross-namespace isolation mechanism. This makes Yama a useful tool for hardening processes within the same environment but not a comprehensive defense against container-to-host process tracing attempts.

The behavior of Yama is controlled by the `/proc/sys/kernel/yama/ptrace_scope` setting, which determines the level of restriction imposed on `ptrace` operations. The possible values are summarized below:

Table 1.1: Yama `ptrace_scope` Settings

Value	**Behavior**
0	No additional restrictions. Processes can trace any other process they have permissions for.
1	Restricted to child processes only. This is the default setting in many distributions.
2	Restricted unless explicitly allowed via `/proc/[pid]/uid_map` or using `prctl()`.
3	No `ptrace` allowed at all, even for processes with `CAP_SYS_PTRACE`. This is the most restrictive setting.

When Yama's ptrace_scope is set to 2, processes cannot freely attach to other unrelated processes, even if they have administrative capabilities or are running as root. In this mode, the ability to trace or debug a process must be explicitly granted by the target process itself or configured through system-level controls.

This permission can be granted using the prctl() system call with the PR_SET_PTRACER option, allowing a specific process to act as a tracer. Alternatively, the system administrator can modify the global ptrace policy by lowering the ptrace_scope value in /proc/sys/kernel/yama/ptrace_scope. Without such explicit authorization, ptrace attempts will consistently fail, regardless of user privileges or capabilities. This design enforces stronger security boundaries by preventing unauthorized inspection or manipulation of processes, even by privileged users, unless the target process explicitly consents to being traced.

Yama is a Linux Security Module (LSM) that enforces additional restrictions on the `ptrace` system call and process debugging mechanisms. Its primary purpose is to reduce the attack surface for exploits that rely on process introspection and manipulation. By limiting the ability of processes to trace or interact with others, Yama enhances system security and makes privilege escalation attacks more difficult. This is particularly valuable in containerized environments and multi-user systems where process isolation is critical.

1.0.8 Kernel Implementation

Yama is implemented in the Linux kernel source file `security/yama/yama_lsm.c`. It integrates with the kernel's LSM framework by registering specific hooks that intercept and evaluate `ptrace` operations. During initialization, Yama registers its hooks using the following function:

```
void __init yama_add_hooks(void)
{
```

```
    pr_info("Yama: becoming mindful.\n");
    security_add_hooks(yama_hooks, ARRAY_SIZE(yama_hooks), "yama");
    yama_init_sysctl();
}
```

Here, `security_add_hooks` registers an array of hook functions defined in `yama_hooks`. These hooks include:

- `ptrace_access_check`
- `ptrace_traceme`
- `task_prctl`
- `task_free`

Each hook is associated with a specific kernel operation, allowing Yama to enforce its security policies at critical points.

1.0.9 Enforcement of `ptrace_scope`

The core of Yama's enforcement logic is implemented in the `yama_ptrace_access_check` function:

```
static int yama_ptrace_access_check(struct task_struct *child,
         unsigned int mode){
}
```

This function determines whether a `ptrace` operation should be permitted based on the current `ptrace_scope` value and the relationship between the tracer and tracee processes.

1.0.10 `ptrace_scope` Settings

Yama provides four levels of `ptrace` restrictions, configurable via `/proc/sys/kernel/yama/ptrace_`

These settings are summarized in Table 1.1.

Chapter 2

Evaluation

2.0.1 Case Study 1: Evaluating the accuracy of CVE-2024-21626

CVE-2024-21626 [1] is a container escape vulnerability in runc, the low-level container runtime used by Docker and other container engines. It allows a malicious container to escape its isolation and execute arbitrary commands on the host system.

The root cause of this vulnerability lies in how runc handles cgroups for containers, which are used to manage system resources like CPU and memory. During container creation, runc opens a file descriptor to the host's `/sys/fs/cgroup` directory to configure resource limits. This descriptor remains open throughout the container's lifecycle and is exposed inside the container via `/proc/self/fd/`, which lists all open file descriptors of the running process. The vulnerability arises when an attacker sets the working directory in the Dockerfile to point to one of these exposed descriptors, such as `/proc/self/fd/N`, where `N` refers to the specific file descriptor inherited from runc that points to the host's cgroup hierarchy. This causes the container's current working directory to resolve to a privileged host path. As a result, all subsequent relative file operations in the container are interpreted from the host's cgroup filesystem rather than the container's own root, allowing unintended access

or modification of host resources.

This behavior is enabled by Linux's treatment of `/proc/self/fd/N` as dynamic symlinks to open file descriptors. When dereferenced, these symlinks resolve to the original host-side path, regardless of container filesystem restrictions. Because runc and Docker do not restrict access to inherited file descriptors or sanitize the working directory path during initialization, attackers can exploit this oversight to escape isolation. By simply changing the container's working directory to a leaked file descriptor, a malicious container can directly interact with host-managed control groups. To test the exploitability of CVE-2024-21626, we designed a Houdini trick that executes the following steps on Docker and runc versions listed in Table 2.1:

1. Set the working directory to `/proc/self/fd/<fd>`, pointing to a controlled file descriptor.

2. Attempt to break out of the container by leveraging the misconfigured working directory.

3. Validate whether commands can be executed on the host system.

The CVE for this vulnerability indicates that the affected runc versions are $\geq$ `1.0.0-rc93`, $\leq$ `1.1.11`. This suggests the vulnerability was introduced in runc version rc93 and fixed in version 1.1.12. However, our tests conducted with Houdini revealed a significant discrepancy with this assertion. Our testing showed that earlier runc versions, such as `1.0.0-rc1`, `1.0.0-rc2`, `1.0.0-rc3`, and `1.0.0-rc4` also exhibited the same vulnerability. This discovery expands the scope of the CVE by revealing that the issue existed well before version `rc93`.

This discrepancy highlights a recurring issue in vulnerability disclosure processes: CVEs often rely on assumptions from static code analysis or limited testing. Without rigorous empirical validation across historical versions, critical edge cases can be missed. In the case of CVE-2024-21626, this oversight could have allowed vulnerable

deployments relying on older runc versions to remain unpatched under the false belief that they were unaffected.

Table 2.1: runc Versions Against CVE-2024-21626

Docker Version	Runc Version	Leaked FD	Exploit Success
1.11.0	runc 0.0.9	-	✗
1.12.0	runc 1.0.0-rc1	/proc/self/fd/4	✓
17.03.0-ce	runc 1.0.0-rc2	/proc/self/fd/4	✓
17.06.1-ce	runc 1.0.0-rc3	/proc/self/fd/5	✓
17.09.0-ce	runc 1.0.0-rc4	/proc/self/fd/5	✓
-	runc 1.0.0-rc8	-	✗
-	runc 1.0.0-rc10	-	✗
-	runc 1.0.0-rc91	-	✗
20.10.17	runc 1.1.2	/proc/self/fd/9	✓
23.0.1	runc 1.1.4	/proc/self/fd/8	✓
23.0.5	runc 1.1.5	/proc/self/fd/8	✓
24.0.2	runc 1.1.7	/proc/self/fd/9	✓
24.0.5	runc 1.1.8	/proc/self/fd/9	✓
24.0.7	runc 1.1.9	/proc/self/fd/9	✓
24.0.0	runc 1.1.11	/proc/self/fd/9	✓
-	runc 1.1.12	-	✗
-	runc 1.1.13	-	✗
-	runc 1.1.14	-	✗

2.0.2 Case Study 2

In this evaluation, we examine whether a Docker container running as a non-privileged user with `UID 1000` can interact with and manipulate host processes that are also owned by `UID 1000`. Under a default Docker configuration, the PID namespace provides process isolation by restricting a container's visibility and access to processes outside its own namespace. However, when the container is launched with the `--pid=host` flag, it shares the host's PID namespace, allowing it to enumerate and interact with host processes. This configuration effectively bypasses one of the core isolation mechanisms provided by containers. To assess the security implications

of this namespace sharing, we investigate whether process-level interactions, such as sending signals or attaching debuggers are permitted between the container and host processes when UID ownership is aligned. The testing environment for this case study is in Table 2.2.

Table 2.2: Case Study 2 Testing Environment

Component	**Specification**
Docker Version	27.0.3
Container Runtime	`runc` 1.1.13

Case study 2's results are summarized in Table **??**, where all pairs of flags were tested using Houdini, and where the cells show what functionality was possible when launching containers with the pair of flags intersecting the rows and columns of Table **??**.

Our objective here is to investigate how various security flags interact with each other, and to determine the minimum set of privileges required for a container to manipulate host processes. Specifically, we looked at the following flags:

- `--pid=host`
- `CAP_KILL`
- `CAP_SYS_PTRACE`
- `--privileged`
- `apparmor=docker-default`

The evaluation began by assessing whether a container launched with `--pid=host` could list host processes using `ps aux`. As expected, the container displayed all processes running on the host, confirming that sharing the host PID namespace fully exposes the host's process table to the container.

Following this, we tested whether a container process could send a `SIGKILL` signal to a host process owned by `UID 1000`. The container was launched with

`--pid=host`, allowing it to see and interact with host processes. Despite not being granted `CAP_KILL`, the signal was successfully delivered. This is because by default, both processes share the same UID and PID namespace, and under standard Linux permission checks, a process can send signals to another process it owns by UID without requiring special capabilities. However, if the target host process is owned by a different UID (another non-root user), `CAP_KILL` is required for the signal to be delivered. That capability must also be effective in the user namespace of the target process. Similarly, signaling host processes owned by `UID 0` remains restricted even with `CAP_KILL`, unless the container process itself is also running as root.

Combining `--pid=host` with `CAP_SYS_PTRACE` enables nearly the same level of interaction with host processes as granting `--privileged` mode. Specifically, it allows a containerized process to list host processes, send signals, and attach a debugger to them, all key capabilities typically associated with privileged containers. However, unlike `--privileged`, this more targeted configuration avoids giving the container unnecessary capabilities such as access to all of the host's devices. These results demonstrate that it is unnecessary to over-privilege containers when finer-grained controls can achieve the same functional outcomes with a significantly better security posture.

When the container is confined by AppArmor's `docker-default` profile, its ability to interact with host processes becomes significantly restricted. Specifically, `docker-default` explicitly denies system calls such as `signal` and `ptrace`, which are required for sending signals and attaching debuggers to other processes. As a result, even if the container shares the PID namespace with the host and possesses the necessary capabilities, these operations will fail.

However, including the `--privileged` flag when starting the container overrides AppArmor confinement by disabling the `docker-default` profile entirely. This lifts all AppArmor-imposed restrictions, allowing the container full access to the previously

restricted system calls. As a result, operations that were previously blocked, such as sending signals or attaching debuggers to host processes are now permitted, even if AppArmor would normally deny them.

It is also important to note that granting capabilities such as `CAP_KILL` and `CAP_SYS_PTRACE` alone is not sufficient for interacting with host processes unless the container shares the host's PID namespace. Without the `--pid=host` flag, the container operates in its own isolated PID namespace, meaning it cannot see or reference host PIDs at all. As a result, any attempt to send signals or attach to host processes will fail.

We note that the explanation above was derived after substantial amount of testing and analysis. The Docker documentation alone does not clarify this behavior. Instead, users are expected to understand both the Docker tooling and the underlying Linux kernel security model to correctly select these flags.

When the container shares the same user namespace as the host (or has aligned UID mappings), and the container process runs under the same UID as the host processes, it can bypass security restrictions that would normally prevent such interactions. In this case, the container may be able to perform operations like sending signals or attaching debuggers to host processes, even without elevated capabilities like `CAP_KILL` or `CAP_SYS_PTRACE`. These capabilities, which are typically required for such actions, are not necessary when the container and host share UID mappings, as the container process is considered the same user as the host processes it targets.

This case study highlights a core strength of Houdini, which is its ability to systematically explore and identify the least privilege for a given operation. By allowing users to declaratively modify container configurations, Houdini makes it easy to test a wide range of privilege combinations in a repeatable and automated way. Instead of relying on trial and error or defaulting to the overly permissive `--privileged` mode, users can incrementally grant permissions and observe the effect of each change.

Houdini thus enables security practitioners, developers, and researchers to precisely determine the minimum set of permissions required for a given trick to succeed. As a result, Houdini not only serves as a testing framework for container escapes and misconfigurations, but also as a practical tool for privilege minimization and hardening container deployments.

This case study highlights the complexity of security policies in containerized environments, highlighting that the runtime behavior of security configurations is not always straightforward or predictable. While administrators may configure a container with `--pid=host` to enable the container to monitor host processes, there is an assumption that certain operations, such as sending signals or attaching debuggers remain restricted. However, these assumptions may not hold at runtime due to the interplay of various factors such as user namespaces and UID mappings.

Table 2.3: Ability for Containers to Interact with Host Processes Across Yama Settings (Ubuntu Kernel 6.8.12)

Security Configuration	Yama Scope	--pid=host	CAP_KILL	CAP_SYS_PTRACE	--privileged	apparmor=docker-default	--pid=host + apparmor=docker-default
--pid=host	0	◇ △	◇ △	◇ △ ⋆	◇ △ ⋆	◇	◇
	1	◇ △	◇ △	◇ △ ⋆	◇ △ ⋆	◇	◇
	2	◇ △	◇ △	◇ △	◇ △	◇	◇
	3	◇ △	◇ △	◇ △	◇ △	◇	◇
CAP_KILL	0	◇ △					◇
	1	◇ △					◇
	2	◇ △					◇
	3	◇ △					◇
CAP_SYS_PTRACE	0	◇ △ ⋆					◇
	1	◇ △ ⋆					◇
	2	◇ △					◇
	3	◇ △					◇
--privileged	0	◇ △ ⋆					◇ △ ⋆
	1	◇ △ ⋆					◇ △ ⋆
	2	◇ △					◇ △
	3	◇ △					◇ △
apparmor=docker-default	0	◇					◇
	1	◇					◇
	2	◇					◇
	3	◇					◇
--pid=host + apparmor=docker-default	0	◇	◇	◇	◇ △ ⋆	◇	◇
	1	◇	◇	◇	◇ △ ⋆	◇	◇
	2	◇	◇	◇	◇ △	◇	◇
	3	◇	◇	◇	◇ △	◇	◇

Table 2.4: *
Tests run as a non-root user (UID=1000) against a host process owned by UID 1000.
◇ List Host Process △ Send Signal to Host Process ⋆ Attach Debugger to Host Process

2.0.3 Case Study 3

To assess the effectiveness of Linux kernel mechanisms in enforcing process isolation within containerized environments, we conducted a series of controlled experiments

targeting the ptrace system call under various configurations. These experiments were designed to evaluate how different kernel versions, Yama security module settings, and process capabilities affect the ability of a containerized process to trace other processes, both inside and outside its container boundary.

Table 2.5: ptrace Behavior Across Distributions with Default Yama Scopes

Distribution	**Kernel Version**	**Yama Scope (Default)**	**CAP_SYS_PTRACE**	**Result**
Debian 12	6.1 (Bookworm)	1	Yes	Success
Debian 12	6.1 (Bookworm)	1	No	Fail
Debian 8	3.16 (Jessie)	0	Yes	Success
Debian 8	3.16 (Jessie)	0	No	Success
Fedora 40	6.7	2	Yes	Fail
Fedora 40	6.7	2	No	Fail
Ubuntu 24.04	6.8.12	1	Yes	Success
Ubuntu 24.04	6.8.12	1	No	Fail
Ubuntu 22.04	5.15 (LTS)	1	Yes	Success
Ubuntu 22.04	5.15 (LTS)	1	No	Fail
Ubuntu 16.04	4.4 (LTS)	0	Yes	Success
Ubuntu 16.04	4.4 (LTS)	0	No	Success
Alpine 3.19	6.1	0	Yes	Success
Alpine 3.19	6.1	0	No	Success
Arch Linux	6.8 (Rolling)	1	Yes	Success
Arch Linux	6.8 (Rolling)	1	No	Fail
CentOS 7	3.10	1	Yes	Success
CentOS 7	3.10	1	No	Fail
CentOS 6	2.6.32	0	Yes	Success
CentOS 6	2.6.32	0	No	Success

Bibliography

[1] Open Containers Maintainers. runc: File descriptor leak leading to container breakout (CVE-2024-21626). `https://github.com/opencontainers/runc/security/advisories/GHSA-xr7r-f8xq-vfvv`, 2024. Accessed: 2024-04-28.